

The Chinese Remainder Theorem

A Lecture: Statement, Proofs, Algorithms, and Applications

Contents

1	Motivation	2
2	Quick-Start Guide: Solving CRT Problems by Hand	2
2.1	What a CRT problem looks like	2
2.2	Method A: Substitution (easiest by hand, no big formulas)	3
2.3	Tool: how to find a modular inverse	5
2.4	Method B: The direct formula (all congruences at once)	6
2.5	Method C: Describe x in two ways and compare	7
2.6	When the moduli are <i>not</i> coprime	10
2.7	Common mistakes	12
2.8	Cheat sheet: which method should I use?	12
2.9	Practice problems (with answers)	12
3	The Two-Modulus Case	13
3.1	Proof 1: Explicit construction via Bézout	13
3.2	Proof 2: Pigeonhole / counting argument	14
4	The General Case: k Pairwise Coprime Moduli	15
5	The Algebraic Formulation: A Ring Isomorphism	16
6	CRT with Non-Coprime Moduli	17
7	The Polynomial Chinese Remainder Theorem	18
8	Applications	18
8.1	Multiplicativity of Euler's totient function	18
8.2	Fast RSA decryption: Garner's algorithm / CRT-RSA	19
8.3	Secret sharing (Asmuth–Bloom scheme)	19
8.4	Computer arithmetic: Residue Number Systems (RNS)	19
8.5	Fast algorithms: multiplying big numbers, FFT-based convolution	19
8.6	Error-correcting codes	19
9	Summary of Key Formulas	20
10	Exercises	20

1 Motivation

The Chinese Remainder Theorem (CRT) answers a question that sounds almost too simple to be deep: *if I tell you the remainder of a number when divided by several different, pairwise coprime numbers, can you tell me the number itself (modulo their product)?* The answer is yes, uniquely, and the reconstruction is completely explicit. This idea dates back at least to Sunzi’s mathematical manual (China, 3rd–5th century CE), which posed a version of the riddle:

There are certain things whose number is unknown. Repeatedly divided by 3, the remainder is 2; by 5, the remainder is 3; by 7, the remainder is 2. What is the number?

We will answer this exact riddle later (Example 4.3). The theorem turns out to be one of the load-bearing pillars of number theory and abstract algebra: it explains why φ is multiplicative, it underlies fast modular arithmetic in cryptography, it powers secret-sharing schemes, and — in its most general algebraic form — it is a statement about how a ring decomposes into a product of simpler rings.

In plain English: think of a number x as being “coded” by its remainders modulo several small numbers. CRT says that if those small numbers share no common factors pairwise, then the code is lossless and invertible: the collection of remainders determines x completely (mod the product), and there’s an explicit recipe to decode it.

Just want to solve problems? Skip straight to Section 2, which contains step-by-step recipes and fully worked examples that need no theory at all.

2 Quick-Start Guide: Solving CRT Problems by Hand

This section is completely self-contained. You do not need any theory or proofs to use it: every step is explained with actual numbers. The later sections explain why these recipes always work.

2.1 What a CRT problem looks like

You are told the *remainders* of an unknown whole number x when it is divided by several numbers, and you must find x :

$$x \equiv a_1 \pmod{m_1}, \quad x \equiv a_2 \pmod{m_2}, \quad \dots, \quad x \equiv a_k \pmod{m_k}.$$

How to read the notation. “ $x \equiv 2 \pmod{3}$ ” simply means “when x is divided by 3, the remainder is 2”. So x could be 2, 5, 8, 11, 14, 17, ... (start at 2 and keep adding 3).

Step 0: check the moduli. Look at every *pair* of moduli. If no two of them share a common factor larger than 1, they are called *pairwise coprime* and everything below works directly.

Moduli	Verdict
3, 5, 7	pairwise coprime
4, 9, 25	pairwise coprime (no shared prime factor)
6, 9	<i>not</i> coprime (both divisible by 3) → see Section 2.6
4, 6, 7	<i>not</i> coprime (4 and 6 share the factor 2) → see Section 2.6

What the answer looks like. Let $M = m_1 m_2 \cdots m_k$ be the product of the moduli. Then there is *exactly one* solution between 0 and $M - 1$, and every other solution is that one plus or minus a multiple of M . We write the complete answer as

$$x \equiv x_0 \pmod{M}.$$

For example, for the moduli 3, 5, 7 we have $M = 105$, and the solutions are $\dots, -82, 23, 128, 233, \dots$ — all of them differ by multiples of 105.

2.2 Method A: Substitution (easiest by hand, no big formulas)

Recipe for Method A

1. Take one congruence, say $x \equiv a \pmod{m}$, and rewrite it as an *equation*:

$$x = a + mt \quad (t \text{ is an unknown whole number}).$$

2. Put this into the next congruence $x \equiv b \pmod{n}$. You get $a + mt \equiv b \pmod{n}$, i.e.

$$mt \equiv b - a \pmod{n}.$$

3. Solve this small congruence for t : you get $t \equiv t_0 \pmod{n}$. (Try values of $t = 0, 1, 2, \dots, n-1$, or multiply by an inverse, see Section 2.3.)
4. Write $t = t_0 + ns$ and put it back:

$$x = a + m(t_0 + ns) = \underbrace{(a + mt_0)}_{\text{new } a} + \underbrace{mn}_{\text{new } m} s.$$

So the two congruences have been merged into the single congruence $x \equiv a + mt_0 \pmod{mn}$.

5. Repeat with the next congruence until none are left.

Tip: it is least work to start with the **largest** modulus.

Example 2.1 (Two congruences, by simply listing numbers). Find x with $x \equiv 2 \pmod{3}$ and $x \equiv 3 \pmod{5}$.

Idea: list the numbers satisfying the *bigger* modulus, and test each one against the other condition.

Numbers with remainder 3 when divided by 5: 3, 8, 13, 18, $\dots$ (add 5 each time).

candidate x	3	8	13
remainder of x divided by 3	0	2	1
what we need	2	2	2
works?	no	yes	no

So $x = 8$. Notice the remainders 0, 2, 1: adding 5 repeatedly cycles through *all* possible remainders mod 3, so you never have to try more than 3 candidates, and exactly one works.

Since $3 \cdot 5 = 15$, the full answer is $x \equiv 8 \pmod{15}$, i.e. 8, 23, 38, $\dots$

Check: $8 = 2 \cdot 3 + 2 \checkmark$ and $8 = 1 \cdot 5 + 3 \checkmark$

Example 2.2 (Method A in symbols, same problem). Solve $x \equiv 2 \pmod{3}$ and $x \equiv 3 \pmod{5}$ with the recipe.

Step 1. Start from the modulus 5: $x = 3 + 5t$.

Step 2. Put into the other congruence: $3 + 5t \equiv 2 \pmod{3}$. Subtract 3: $5t \equiv -1 \pmod{3}$.

Step 3. Solve $5t \equiv -1 \pmod{3}$ for t . We do it in three small moves.

- (a) **Make the left number small.** Dividing 5 by 3 leaves remainder 2, so $5 \equiv 2 \pmod{3}$. Hence $5t$ behaves like $2t$, and the congruence becomes $2t \equiv -1 \pmod{3}$.
- (b) **Make the right number positive.** Adding the modulus 3 does not change a remainder, so $-1 \equiv -1 + 3 = 2 \pmod{3}$. The congruence becomes $2t \equiv 2 \pmod{3}$.
- (c) **Find t .** Ask: which t makes $2t$ leave remainder 2 when divided by 3? Just try the possible remainders $t = 0, 1, 2$:

t	0	1	2
$2t$	0	2	4
remainder of $2t$ divided by 3	0	2	1
equals 2?	no	yes	no

Only $t = 1$ works, so $t \equiv 1 \pmod{3}$, i.e. $t \in \{1, 4, 7, \dots\}$.

(Shortcut: since 2 and 3 share no factor, you may divide both sides of $2t \equiv 2$ by 2 and get $t \equiv 1$. The table above shows this is safe.)

Step 4. Write $t = 1 + 3s$: $x = 3 + 5(1 + 3s) = 8 + 15s$.

Answer: $x \equiv 8 \pmod{15}$.

Example 2.3 (Sunzi's riddle by Method A: three congruences). Solve

$$x \equiv 2 \pmod{3}, \quad x \equiv 3 \pmod{5}, \quad x \equiv 2 \pmod{7}.$$

Here $M = 3 \cdot 5 \cdot 7 = 105$. Start with the largest modulus, 7.

Round 1: bring in the modulus 5.

$$\begin{array}{ll}
 x = 2 + 7t & \text{(from } x \equiv 2 \pmod{7}\text{)} \\
 2 + 7t \equiv 3 \pmod{5} & \text{(the second condition)} \\
 7t \equiv 1 \pmod{5} & \text{(subtract 2)} \\
 2t \equiv 1 \pmod{5} & (7 \equiv 2 \pmod{5}) \\
 t \equiv 3 \pmod{5} & \text{(because } 2 \cdot 3 = 6 \equiv 1)
 \end{array}$$

So $t = 3 + 5s$ and $x = 2 + 7(3 + 5s) = 23 + 35s$. We have merged two conditions:

$$x \equiv 23 \pmod{35}.$$

Round 2: bring in the modulus 3.

$$\begin{array}{ll}
 x = 23 + 35s & \\
 23 + 35s \equiv 2 \pmod{3} & \text{(the last condition)} \\
 2 + 2s \equiv 2 \pmod{3} & (23 \equiv 2, 35 \equiv 2 \pmod{3}) \\
 2s \equiv 0 \pmod{3} & \\
 s \equiv 0 \pmod{3} &
 \end{array}$$

So $s = 3r$ and $x = 23 + 35 \cdot 3r = 23 + 105r$.

Answer: $x \equiv 23 \pmod{105}$. The smallest positive solution is $x = 23$.

Check: $23 = 7 \cdot 3 + 2 \checkmark$; $23 = 4 \cdot 5 + 3 \checkmark$; $23 = 3 \cdot 7 + 2 \checkmark$

Example 2.4 (A harder one: moduli 4, 9, 25). Solve $x \equiv 1 \pmod{4}$, $x \equiv 2 \pmod{9}$, $x \equiv 3 \pmod{25}$. Here $M = 4 \cdot 9 \cdot 25 = 900$.

Round 1. Start with 25: $x = 3 + 25t$. Put into the condition mod 9:

$$\begin{aligned} 3 + 25t &\equiv 2 \pmod{9} \\ 25t &\equiv -1 \pmod{9} \\ 7t &\equiv 8 \pmod{9} \end{aligned} \qquad (25 \equiv 7, -1 \equiv 8)$$

We need a number that turns 7 into 1 modulo 9: try $7 \cdot 1 = 7$, $7 \cdot 2 = 14 \equiv 5$, $7 \cdot 3 = 21 \equiv 3$, $7 \cdot 4 = 28 \equiv 1$. So multiply both sides by 4:

$$t \equiv 8 \cdot 4 = 32 \equiv 5 \pmod{9}.$$

Thus $t = 5 + 9s$, and $x = 3 + 25(5 + 9s) = 128 + 225s$. So $x \equiv 128 \pmod{225}$.

Round 2. Put $x = 128 + 225s$ into the condition mod 4:

$$\begin{aligned} 128 + 225s &\equiv 1 \pmod{4} \\ 0 + 1 \cdot s &\equiv 1 \pmod{4} & (128 \equiv 0, 225 \equiv 1 \pmod{4}) \\ s &\equiv 1 \pmod{4} \end{aligned}$$

So $s = 1 + 4r$ and $x = 128 + 225(1 + 4r) = 353 + 900r$.

Answer: $x \equiv 353 \pmod{900}$.

Check: $353 = 88 \cdot 4 + 1 \checkmark$; $353 = 39 \cdot 9 + 2 \checkmark$; $353 = 14 \cdot 25 + 3 \checkmark$

2.3 Tool: how to find a modular inverse

Methods B and C below need a number called an *inverse*.

Definition 2.5 (Inverse, in plain words). The inverse of c modulo n is a number y such that $c \cdot y$ leaves remainder 1 when divided by n . We write $y \equiv c^{-1} \pmod{n}$. It exists exactly when c and n share no common factor (this is why CRT needs coprime moduli).

Way 1: reduce, then try small multiples (best for small n). First replace c by its remainder mod n . Then compute $c, 2c, 3c, \dots \pmod{n}$ until you see 1.

Example 2.6 (Inverse of 7 modulo 9). $7 \cdot 1 = 7$, $7 \cdot 2 = 14 \equiv 5$, $7 \cdot 3 = 21 \equiv 3$, $7 \cdot 4 = 28 \equiv 1$. So $7^{-1} \equiv 4 \pmod{9}$.

Example 2.7 (Inverse of 35 modulo 4). First reduce: $35 = 8 \cdot 4 + 3$, so $35 \equiv 3$. Then $3 \cdot 1 = 3$, $3 \cdot 2 = 6 \equiv 2$, $3 \cdot 3 = 9 \equiv 1$. So $35^{-1} \equiv 3 \pmod{4}$.

Way 2: Euclid's algorithm (for larger numbers). Divide repeatedly until the remainder is 1, then substitute backwards.

Example 2.8 (Inverse of 36 modulo 25). Reduce first: $36 \equiv 11 \pmod{25}$, so we need the inverse of 11.

Forward:

$$25 = 2 \cdot 11 + 3, \quad 11 = 3 \cdot 3 + 2, \quad 3 = 1 \cdot 2 + 1.$$

Backward (solve each line for its remainder and substitute):

$$\begin{aligned} 1 &= 3 - 1 \cdot 2 \\ &= 3 - 1 \cdot (11 - 3 \cdot 3) = 4 \cdot 3 - 1 \cdot 11 \\ &= 4 \cdot (25 - 2 \cdot 11) - 1 \cdot 11 = 4 \cdot 25 - 9 \cdot 11. \end{aligned}$$

Reading this modulo 25: $1 \equiv -9 \cdot 11$, so $11^{-1} \equiv -9 \equiv 16 \pmod{25}$.

Check: $36 \cdot 16 = 576 = 23 \cdot 25 + 1 \checkmark$

2.4 Method B: The direct formula (all congruences at once)

Recipe for Method B

$$x \equiv \sum_{i=1}^k a_i M_i y_i \pmod{M} \quad \text{where} \quad M = m_1 \cdots m_k, \quad M_i = \frac{M}{m_i}, \quad M_i y_i \equiv 1 \pmod{m_i}.$$

1. Compute M , the product of all moduli.
2. For each i , compute $M_i = M/m_i$ (= the product of all the *other* moduli).
3. For each i , find y_i , the inverse of M_i modulo m_i (reduce $M_i \bmod m_i$ first!).
4. Compute the sum $a_1 M_1 y_1 + a_2 M_2 y_2 + \cdots + a_k M_k y_k$.
5. Divide the sum by M and keep the **remainder**. That is x_0 .

Why it works: each term is one “component” of x . Think of x as being stored by its list of remainders, like the coordinates of a vector. For Sunzi’s riddle below, x is the list $(2, 3, 2)$ (remainders mod 3, 5, 7), and we can split it into coordinates:

$$(2, 3, 2) = 2 \cdot (1, 0, 0) + 3 \cdot (0, 1, 0) + 2 \cdot (0, 0, 1).$$

The number $M_i y_i$ is the integer that plays the role of the “unit vector” for position i : it leaves remainder 1 when divided by m_i and remainder 0 when divided by every other modulus. It is built in two moves:

- M_i is the product of all the *other* moduli, so every other m_j divides it. Hence M_i (and every multiple of it) leaves remainder 0 for all m_j with $j \neq i$.
- M_i itself leaves some remainder at m_i , not necessarily 1. Multiplying by y_i , the inverse of M_i modulo m_i , turns that remainder into exactly 1, and it does not destroy the zeros elsewhere (multiples of M_i stay multiples of M_i).

Multiplying by a_i scales the “1” up to the wanted remainder a_i . So the term $a_i M_i y_i$ is the a_i -*component* of x : it has remainder a_i at m_i and remainder 0 at every other modulus. Adding all the components rebuilds the whole list of remainders, and therefore x . For Sunzi’s riddle:

term	value	mod 3	mod 5	mod 7
$a_1 M_1 y_1 = 2 \cdot 35 \cdot 2$	140	2	0	0
$a_2 M_2 y_2 = 3 \cdot 21 \cdot 1$	63	0	3	0
$a_3 M_3 y_3 = 2 \cdot 15 \cdot 1$	30	0	0	2
sum	233	2	3	2

Every column has exactly one non-zero entry, and it is exactly the remainder we wanted.

Why “mod M ” at the end? A component is not “a piece of M ” in size (here $140 > 105$); it is a component in the sense of *remainders*. The sum 233 is a valid solution, but so is $233 - 105 = 128$ or $233 - 210 = 23$, because subtracting $M = m_1 m_2 \cdots m_k$ changes none of the remainders (it is a multiple of every m_i). We keep the smallest non-negative one.

Example 2.9 (Sunzi's riddle by Method B). $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 2 \pmod{7}$.

Step 1: $M = 3 \cdot 5 \cdot 7 = 105$.

Steps 2–4: fill in a table.

i	m_i	a_i	$M_i = 105/m_i$	$M_i \bmod m_i$	y_i (inverse)	$a_i M_i y_i$
1	3	2	35	$35 = 11 \cdot 3 + 2 \rightarrow 2$	2 ($2 \cdot 2 = 4 \equiv 1$)	$2 \cdot 35 \cdot 2 = 140$
2	5	3	21	$21 = 4 \cdot 5 + 1 \rightarrow 1$	1 ($1 \cdot 1 = 1$)	$3 \cdot 21 \cdot 1 = 63$
3	7	2	15	$15 = 2 \cdot 7 + 1 \rightarrow 1$	1 ($1 \cdot 1 = 1$)	$2 \cdot 15 \cdot 1 = 30$

Step 5: sum = $140 + 63 + 30 = 233$. Divide by 105: $233 = 2 \cdot 105 + 23$. So

$$x \equiv 23 \pmod{105}.$$

Example 2.10 (Another one with less friendly numbers). $x \equiv 3 \pmod{4}$, $x \equiv 2 \pmod{5}$, $x \equiv 1 \pmod{7}$.

$$M = 4 \cdot 5 \cdot 7 = 140.$$

i	m_i	a_i	M_i	$M_i \bmod m_i$	y_i	$a_i M_i y_i$
1	4	3	35	3	3 ($3 \cdot 3 = 9 \equiv 1$)	$3 \cdot 35 \cdot 3 = 315$
2	5	2	28	3	2 ($3 \cdot 2 = 6 \equiv 1$)	$2 \cdot 28 \cdot 2 = 112$
3	7	1	20	6	6 ($6 \cdot 6 = 36 \equiv 1$)	$1 \cdot 20 \cdot 6 = 120$

Sum = $315 + 112 + 120 = 547$, and $547 = 3 \cdot 140 + 127$. So

$$x \equiv 127 \pmod{140}.$$

Check: $127 = 31 \cdot 4 + 3 \checkmark$; $127 = 25 \cdot 5 + 2 \checkmark$; $127 = 18 \cdot 7 + 1 \checkmark$

Remark 2.11 (A ready-made formula for the moduli 3, 5, 7). For these three moduli the numbers $M_i y_i$ are always the same: 70, 21, 15. Indeed 70 leaves remainders (1, 0, 0) when divided by (3, 5, 7), 21 leaves (0, 1, 0), and 15 leaves (0, 0, 1). Hence

$$\boxed{x \equiv 70a + 21b + 15c \pmod{105}} \quad \text{for } x \equiv a \pmod{3}, x \equiv b \pmod{5}, x \equiv c \pmod{7}.$$

Party trick: ask a friend to think of a number from 1 to 105 and tell you its remainders after dividing by 3, 5, 7. Say they answer 1, 3, 2. Then $70 \cdot 1 + 21 \cdot 3 + 15 \cdot 2 = 70 + 63 + 30 = 163$, and $163 - 105 = 58$. Their number is 58. (Check: $58 = 19 \cdot 3 + 1$, $58 = 11 \cdot 5 + 3$, $58 = 8 \cdot 7 + 2$.)

The same idea works for *any* fixed set of moduli: compute the numbers $M_i y_i$ once, and reuse them for every problem.

2.5 Method C: Describe x in two ways and compare

Method C solves **two** congruences

$$x \equiv a \pmod{m}, \quad x \equiv b \pmod{n}, \quad \gcd(m, n) = 1.$$

There is no formula to memorize: the answer comes out of one simple idea. (This is also the method used in practice, for instance in RSA decryption, because all its numbers stay small.)

The idea: “start at the remainder, then take steps”

What does $x \equiv a \pmod{m}$ say? That x is a plus some number of whole steps of size m :

$$x = a + m \cdot t \quad (t = 0, 1, 2, \dots \text{ counts the steps}).$$

For example, “remainder 2 when divided by 3” means $x \in \{2, 2 + 3, 2 + 6, 2 + 9, \dots\} = \{2, 5, 8, 11, \dots\}$.

In exactly the same way, $x \equiv b \pmod{n}$ says that x is b plus some number of steps of size n :

$$x = b + n \cdot s.$$

But x is *one* number, so both descriptions must give the same value:

$$a + m t = b + n s.$$

Our only unknowns are the step counts t and s . Now comes the one clever move: **look at both sides through the eyes of n** , i.e. look only at remainders when dividing by n . Then $n s$ is a multiple of n and simply *disappears*, leaving

$$a + m t \equiv b \pmod{n} \iff m t \equiv b - a \pmod{n}.$$

In words: *the steps of size m must add up to exactly the gap $b - a$, as seen by n* . Find t from this, and the answer is $x = a + m t$.

Guided example: $x \equiv 2 \pmod{3}$ and $x \equiv 3 \pmod{5}$

Here $a = 2$, $m = 3$, $b = 3$, $n = 5$. Every number of the form $x = 2 + 3t$ already has remainder 2 when divided by 3. We only have to choose t so that the remainder when divided by 5 is 3. Just try the first few values:

number of steps t	0	1	2	3	4
$x = 2 + 3t$	2	5	8	11	14
remainder of x divided by 5	2	0	3	1	4

We want remainder 3, and this happens for $t = 2$. So $x = 2 + 3 \cdot 2 = 8$. Notice that the remainders 2, 0, 3, 1, 4 contain every value from 0 to 4 exactly once: since 3 and 5 share no factor, the steps of size 3 visit every remainder mod 5, so exactly one t between 0 and 4 works. Since the pattern repeats after $3 \cdot 5 = 15$, the full answer is $x \equiv 8 \pmod{15}$.

When the numbers are big: let the inverse find t for you

Trying $t = 0, 1, 2, \dots$ is fine for tiny numbers but hopeless for large ones. Look again at what we had to solve:

$$m t \equiv b - a \pmod{n} \quad (\text{above: } 3t \equiv 1 \pmod{5}).$$

If we had a number that “undoes” multiplication by m , we could just apply it to both sides. That number is the *inverse* m^{-1} of m modulo n : it satisfies $m \cdot m^{-1} \equiv 1 \pmod{n}$ (see Section 2.3 for how to find it). Multiplying both sides by it gives t directly:

$$t \equiv (b - a) \cdot m^{-1} \pmod{n}, \quad \text{and then} \quad x = a + m t \pmod{mn}.$$

In the example, $3 \cdot 2 = 6 \equiv 1 \pmod{5}$, so $3^{-1} = 2$ and $t \equiv (3 - 2) \cdot 2 = 2$. It is the same $t = 2$ as in the table, obtained without trying anything.

Why the answer works, in two lines. Modulo m : $x = a + m t$ differs from a by a multiple of m , so the remainder is a , whatever t is. Modulo n : $m t \equiv (b - a) m m^{-1} \equiv b - a$, so $x \equiv a + (b - a) = b$.

The formula, stated formally

Everything above can be condensed into one statement.

Theorem 2.12 (Two-congruence formula (Method C)). *Let $m, n \geq 2$ be integers with $\gcd(m, n) = 1$, and let a, b be any integers. Let u be an inverse of m modulo n , i.e. an integer with $mu \equiv 1 \pmod{n}$. Then the system*

$$x \equiv a \pmod{m}, \quad x \equiv b \pmod{n}$$

has exactly one solution modulo mn , namely

$$x \equiv a + mt \pmod{mn}, \quad \text{where } t \equiv (b - a)u \pmod{n}.$$

Equivalently, in “component” form (compare Method B),

$$x \equiv a(1 - mu) + b(mu) \pmod{mn}.$$

Proof. It is a solution. Modulo m : $x - a = mt$ is a multiple of m , so $x \equiv a$. Modulo n : $mt \equiv mu(b - a) \equiv b - a$, so $x \equiv a + (b - a) = b$.

It is the only one. If x and x' are both solutions, then $m \mid x - x'$ and $n \mid x - x'$. Because $\gcd(m, n) = 1$, this forces $mn \mid x - x'$, i.e. $x \equiv x' \pmod{mn}$.

Component form. Since $t \equiv u(b - a) \pmod{n}$, we can write $t = u(b - a) + nk$ for some integer k . Then $mt = mu(b - a) + mnk \equiv mu(b - a) \pmod{mn}$, so

$$x \equiv a + mu(b - a) = a(1 - mu) + b(mu) \pmod{mn}.$$

□

Symbols at a glance.

Symbol	Meaning
a, m	remainder and modulus of the first congruence
b, n	remainder and modulus of the second congruence
$u = m^{-1} \pmod{n}$	the number with $mu \equiv 1 \pmod{n}$
$b - a$	the gap the steps of size m must cover, as seen by n
t	the number of steps of size m taken from a
mn	the modulus of the final answer

Example 2.13 (RSA and the one-to-one mapping: $p = 7$, $q = 11$, negative gap). Take the tiny RSA modulus $N = pq = 7 \cdot 11 = 77$, with public exponent $e = 7$ and private exponent $d = 43$ (indeed $ed = 301 = 5 \cdot 60 + 1$ and $\varphi(77) = 6 \cdot 10 = 60$).

Why CRT is the right tool: a one-to-one mapping. Every message $x \in \{0, 1, \dots, 76\}$ is described *uniquely* by the pair of remainders $(x \bmod 7, x \bmod 11)$. There are 77 messages and $7 \cdot 11 = 77$ pairs, and two different messages never give the same pair, because the two-congruence formula (Theorem 2.12) returns exactly one solution modulo 77 for each pair. So

$$x \longleftrightarrow (x \bmod 7, x \bmod 11)$$

is a one-to-one correspondence, and RSA can be computed on the pair instead of on x .

Message to pair. Let the message be $x = 68$. Then $68 = 9 \cdot 7 + 5$ and $68 = 6 \cdot 11 + 2$, so $68 \leftrightarrow (5, 2)$.

Encryption, component by component. Raising to the power $e = 7$ works separately in each coordinate:

$$5^7 \equiv 5 \pmod{7}, \quad 2^7 = 128 \equiv 7 \pmod{11}.$$

So the ciphertext corresponds to the pair $(5, 7)$. To turn it into a number, use the two-congruence formula with $a = 5$, $m = 7$, $b = 7$, $n = 11$. Since $7 \cdot 8 = 56 \equiv 1 \pmod{11}$, we have $u = 7^{-1} = 8$, and

$$t \equiv (b - a)u = (7 - 5) \cdot 8 = 16 \equiv 5 \pmod{11}, \quad c = 5 + 7 \cdot 5 = 40.$$

So $c = 40$. (Check: $68^7 \equiv 40 \pmod{77}$.)

Decryption with CRT, and a negative gap. Reduce the private exponent for each prime, using Fermat: $d_p = 43 \bmod 6 = 1$ and $d_q = 43 \bmod 10 = 3$. Then

$$m_p = 40^1 \equiv 5 \pmod{7}, \quad m_q = 40^3 \equiv 7^3 = 343 \equiv 2 \pmod{11}.$$

We now need x with $x \equiv 5 \pmod{7}$ and $x \equiv 2 \pmod{11}$. Here $a = 5$, $m = 7$, $b = 2$, $n = 11$, so the gap is negative: $b - a = 2 - 5 = -3$. Write $x = 5 + 7t$; we need $7t \equiv -3 \pmod{11}$. The inverse is taken modulo the *other* modulus: $u = 7^{-1} = 8$ modulo 11, as $7 \cdot 8 = 56 = 5 \cdot 11 + 1$. Then

$$t \equiv (-3) \cdot 8 = -24 \equiv -24 + 33 = 9 \pmod{11}, \quad x = 5 + 7 \cdot 9 = 68.$$

The formula returns the original message, $x \equiv 68 \pmod{77}$. This is precisely the one-to-one correspondence at work: the pair $(5, 2)$ can only come from 68.

Check: $68 = 9 \cdot 7 + 5$ ✓ $68 = 6 \cdot 11 + 2$ ✓ and direct decryption $40^{43} \equiv 68 \pmod{77}$ ✓

Example 2.14 (Three congruences: Sunzi again, one merge at a time). Solve $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 2 \pmod{7}$.

First merge the first two. We just found $x \equiv 8 \pmod{15}$. This single congruence replaces the first two.

Now merge it with the third. Take $a = 8$, $m = 15$ (so $x = 8 + 15t$) and $b = 2$, $n = 7$. We need $8 + 15t \equiv 2 \pmod{7}$, i.e. $15t \equiv -6 \pmod{7}$. Since $15 = 2 \cdot 7 + 1$, we have $15 \equiv 1$, so this is simply $t \equiv -6 \equiv 1 \pmod{7}$.

Hence $x = 8 + 15 \cdot 1 = 23$, and $x \equiv 23 \pmod{105}$. This agrees with Method B.

Example 2.15 (Method C inside RSA decryption). In CRT-RSA decryption with $p = 61$, $q = 53$ one has computed $m_q = 12$ and $m_p = 4$ and wants the message m with $m \equiv 12 \pmod{53}$ and $m \equiv 4 \pmod{61}$.

Take $a = 12$, $m = 53$ (so $x = 12 + 53t$) and $b = 4$, $n = 61$. We need $12 + 53t \equiv 4 \pmod{61}$, i.e. $53t \equiv -8 \pmod{61}$.

The inverse of 53 modulo 61 is 38, because $53 \cdot 38 = 2014 = 33 \cdot 61 + 1$. (In RSA this number is precomputed and called q_{inv} .) So

$$t \equiv (-8) \cdot 38 = -304 \equiv -304 + 305 = 1 \pmod{61}.$$

(In RSA this t is called h .) Hence $x = 12 + 53 \cdot 1 = 65$: the decrypted message is 65.

Remark 2.16 (Connection with Method B). The number $e_2 := m \cdot m^{-1}$ leaves remainder 1 at n and 0 at m : it is the “unit component” for n in the language of Method B. Then $1 - e_2$ is the unit component for m , and

$$x = a + (b - a) e_2 = a(1 - e_2) + b e_2,$$

which is “ $a \cdot (\text{component}_1) + b \cdot (\text{component}_2)$ ”. So Method C is Method B for two moduli, with one advantage: the second unit comes for free as $1 - e_2$, so only one inverse is needed.

Common slip	Correct version
Inverting m modulo m (impossible)	invert m modulo the <i>other</i> modulus n
Forgetting to reduce t	reduce t modulo n before computing $a + mt$
Reporting the answer modulo m or n	the answer is modulo mn

2.6 When the moduli are *not* coprime

Sometimes two moduli share a factor (e.g. 6 and 9 share 3). Then the system may have *no* solution, or the solution repeats more often than mn .

Recipe for non-coprime moduli

For $x \equiv a \pmod{m}$ and $x \equiv b \pmod{n}$, let $g = \gcd(m, n)$.

1. **Consistency test:** compare the remainders of a and b after dividing by g . If $a \bmod g \neq b \bmod g$, there is **no solution**. Stop.
2. Otherwise use Method A: write $x = a + mt$, giving $mt \equiv b - a \pmod{n}$.
3. **Divide everything by g** — the left side, the right side *and the modulus*:

$$\frac{m}{g} t \equiv \frac{b - a}{g} \pmod{\frac{n}{g}}.$$

Now m/g and n/g are coprime, so you can solve for t as usual.

4. The final answer is a congruence modulo $\text{lcm}(m, n) = \frac{mn}{g}$ (not modulo mn).

Example 2.17 (A solvable non-coprime system). $x \equiv 4 \pmod{6}$ and $x \equiv 1 \pmod{9}$.

Test: $g = \gcd(6, 9) = 3$. Remainders mod 3: $4 \rightarrow 1$ and $1 \rightarrow 1$. They agree, so a solution exists.

Solve: $x = 4 + 6t$. Then $4 + 6t \equiv 1 \pmod{9}$, so $6t \equiv -3 \pmod{9}$. Divide everything by 3 (including the modulus $9 \rightarrow 3$):

$$2t \equiv -1 \equiv 2 \pmod{3} \implies t \equiv 1 \pmod{3}.$$

So $t = 1 + 3s$, $x = 4 + 6(1 + 3s) = 10 + 18s$.

Answer: $x \equiv 10 \pmod{18}$, since $\text{lcm}(6, 9) = 18$.

Check: $10 = 1 \cdot 6 + 4 \checkmark$; $10 = 1 \cdot 9 + 1 \checkmark$

Example 2.18 (A system with no solution). $x \equiv 2 \pmod{4}$ and $x \equiv 1 \pmod{6}$.

$g = \gcd(4, 6) = 2$. Remainders mod 2: $2 \rightarrow 0$ but $1 \rightarrow 1$. They disagree, so **no such x exists**.

Common-sense reason: the first condition says x is *even* (it is 2, 6, 10, ...), and the second says x is *odd* (1, 7, 13, ...). No number is both.

Example 2.19 (Redundant conditions: the basket of eggs). *A basket of eggs: if you take them out 2, 3, 4, 5 or 6 at a time, exactly one egg is left over. If you take them out 7 at a time, none are left. What is the smallest possible number of eggs?*

Simplify first. “One left over when taking out 2, 3, 4, 5 or 6 at a time” means that $x - 1$ is divisible by all of 2, 3, 4, 5, 6, i.e. by their lcm = 60. So the five conditions collapse into one: $x \equiv 1 \pmod{60}$. Together with $x \equiv 0 \pmod{7}$ we have a coprime system ($\gcd(60, 7) = 1$).

Solve with Method A: $x = 1 + 60t$ and $1 + 60t \equiv 0 \pmod{7}$. Since $60 = 8 \cdot 7 + 4$:

$$4t \equiv -1 \equiv 6 \pmod{7}.$$

The inverse of 4 mod 7 is 2 ($4 \cdot 2 = 8 \equiv 1$), so $t \equiv 6 \cdot 2 = 12 \equiv 5 \pmod{7}$. Hence $x = 1 + 60 \cdot 5 = 301$.

Answer: the smallest number is 301 eggs (in general $x \equiv 301 \pmod{420}$). *Check:* $301 = 7 \cdot 43$ ✓ and $301 - 1 = 300 = 5 \cdot 60$ ✓

Example 2.20 (Word problem: when do two schedules meet?). *Ana swims every 6 days and her next swim is in 4 days (so on days 4, 10, 16, ...). Ben swims every 9 days and his next swim is in 1 day (days 1, 10, 19, ...). When do they first swim on the same day, and how often after that?*

In symbols: $x \equiv 4 \pmod{6}$ and $x \equiv 1 \pmod{9}$. This is exactly Example 2.17: the answer is $x \equiv 10 \pmod{18}$.

They first meet on **day 10**, and then every 18 days: days 10, 28, 46, ...

2.7 Common mistakes

- **Not checking coprimality.** Methods B and C need pairwise coprime moduli. If two moduli share a factor, use the recipe of Section 2.6.
- **Wrong inverse in Method B.** y_i is the inverse of M_i modulo m_i — not modulo M , and not the inverse of m_i .
- **Forgetting to reduce first.** Before searching for an inverse, replace M_i by $M_i \bmod m_i$. It makes the numbers small.
- **Negative numbers.** If you get -82 , add M until you land between 0 and $M - 1$: $-82 + 105 = 23$.
- **Dividing a congruence without dividing the modulus.** In $6t \equiv 6 \pmod{9}$ you may divide by 3, but then the modulus becomes 3: $2t \equiv 2 \pmod{3}$. (Dividing to get $t \equiv 1 \pmod{9}$ would lose solutions.)
- **Skipping the check.** Always test your final x in each original congruence. It takes ten seconds and catches almost every arithmetic slip.

2.8 Cheat sheet: which method should I use?

Situation	Best tool
Two small moduli	List numbers $a, a+m, a+2m, \dots$ and test (Example 2.1)
Several congruences, by hand	Method A: substitution, largest modulus first (Examples 2.3, 2.4)
Same moduli used repeatedly, or writing a program	Method B: $x \equiv \sum a_i M_i y_i \pmod{M}$ (Examples 2.9, 2.10)
Merging two at a time (programming)	Method C: $x \equiv a + m[(b-a)m^{-1} \bmod n] \pmod{mn}$
Moduli share a factor	Consistency test, divide by g , answer modulo the lcm (Examples 2.17, 2.18)
Many conditions like “remainder 1 for 2, 3, 4, 5, 6”	Collapse them to one congruence modulo the lcm (Example 2.19)

2.9 Practice problems (with answers)

Try each one on paper first; answers are in the table below.

P1. $x \equiv 1 \pmod{3}$, $x \equiv 2 \pmod{5}$.

P2. $x \equiv 3 \pmod{4}$, $x \equiv 4 \pmod{7}$.

P3. $x \equiv 2 \pmod{3}$, $x \equiv 1 \pmod{4}$, $x \equiv 3 \pmod{5}$.

P4. Using the 3, 5, 7 formula: $x \equiv 1 \pmod{3}$, $x \equiv 4 \pmod{5}$, $x \equiv 6 \pmod{7}$.

P5. $x \equiv 6 \pmod{10}$, $x \equiv 2 \pmod{8}$.

P6. $x \equiv 1 \pmod{6}$, $x \equiv 2 \pmod{9}$.

Answer	Quick check
P1 $x \equiv 7 \pmod{15}$	$7 = 2 \cdot 3 + 1$, $7 = 1 \cdot 5 + 2$
P2 $x \equiv 11 \pmod{28}$	$11 = 2 \cdot 4 + 3$, $11 = 1 \cdot 7 + 4$
P3 $x \equiv 53 \pmod{60}$	$53 = 17 \cdot 3 + 2$, $53 = 13 \cdot 4 + 1$, $53 = 10 \cdot 5 + 3$
P4 $x \equiv 34 \pmod{105}$	$70 + 84 + 90 = 244 = 2 \cdot 105 + 34$; $34 = 11 \cdot 3 + 1$, $34 = 6 \cdot 5 + 4$, $34 = 4 \cdot 7 + 6$
P5 $x \equiv 26 \pmod{40}$	$\gcd = 2$, both remainders even; $26 = 2 \cdot 10 + 6$, $26 = 3 \cdot 8 + 2$
P6 no solution	$\gcd = 3$, but $1 \bmod 3 = 1 \neq 2 = 2 \bmod 3$

3 The Two-Modulus Case

Theorem 3.1 (CRT, two moduli). *Let m, n be positive integers with $\gcd(m, n) = 1$. Then for any integers a, b , the system*

$$x \equiv a \pmod{m}, \quad x \equiv b \pmod{n}$$

has a solution $x \in \mathbb{Z}$, and this solution is unique modulo mn . That is, there exists x_0 with $0 \leq x_0 < mn$ solving the system, and the full solution set is $\{x_0 + kmn : k \in \mathbb{Z}\}$.

We prove this in two independent ways: first by an explicit construction using Bézout's identity (this is the version you would program on a computer), and second by a slicker counting/pigeonhole argument (this is the version that generalizes best conceptually).

3.1 Proof 1: Explicit construction via Bézout

Lemma 3.2 (Bézout's identity). *If $\gcd(m, n) = 1$, there exist integers u, v such that*

$$mu + nv = 1.$$

Proof. This is a standard consequence of the Euclidean algorithm: running the algorithm on (m, n) to compute $\gcd(m, n)$ and then back-substituting each division step expresses the final gcd, 1, as an integer combination $mu + nv$. (See any elementary number theory text for the full back-substitution; we take this as known machinery.) $\square$

Proof of Theorem 3.1, construction. By Lemma 3.2, choose integers u, v with

$$mu + nv = 1.$$

Define

$$x_0 = anv + bmu.$$

Check the first congruence. Since $mu = 1 - nv$, we have $bmu = b(1 - nv) = b - bnv$, so

$$x_0 = anv + b - bnv = b + (a - b)nv.$$

Hmm — let's instead check directly and cleanly. Modulo m : since $mu \equiv 0 \pmod{m}$, we get $nv \equiv 1 - mu \equiv 1 \pmod{m}$. Therefore

$$x_0 = a(nv) + b(mu) \equiv a \cdot 1 + b \cdot 0 = a \pmod{m}.$$

Check the second congruence. Modulo n : since $nv \equiv 0 \pmod{n}$, we get $mu \equiv 1 - nv \equiv 1 \pmod{n}$. Therefore

$$x_0 = a(nv) + b(mu) \equiv a \cdot 0 + b \cdot 1 = b \pmod{n}.$$

So x_0 solves both congruences simultaneously. This proves **existence**.

Uniqueness modulo mn . Suppose x_0 and x_1 both solve the system. Then

$$x_0 \equiv x_1 \pmod{m} \quad \text{and} \quad x_0 \equiv x_1 \pmod{n},$$

i.e. $m \mid (x_0 - x_1)$ and $n \mid (x_0 - x_1)$. Since $\gcd(m, n) = 1$, this forces $mn \mid (x_0 - x_1)$ (a standard fact: if two coprime numbers each divide D , their product divides D — this itself follows from unique factorization, since m and n share no prime factors, so all prime-power factors of m and of n must appear in D simultaneously). Hence $x_0 \equiv x_1 \pmod{mn}$. $\square$

In plain English: Bézout’s identity gives us two special “indicator” numbers: $e_1 = nv$ is $\equiv 1 \pmod{m}$ but $\equiv 0 \pmod{n}$, and $e_2 = mu$ is $\equiv 0 \pmod{m}$ but $\equiv 1 \pmod{n}$. These act like a basis: to hit target remainder $a \pmod{m}$ and $b \pmod{n}$ simultaneously, just take the combination $a \cdot e_1 + b \cdot e_2$ — each term contributes exactly where it’s supposed to and vanishes where it isn’t supposed to interfere. This “indicator function” idea is exactly the same trick used in Lagrange interpolation (see Section 7).

3.2 Proof 2: Pigeonhole / counting argument

Proof of Theorem 3.1, counting. Consider the function

$$f : \{0, 1, \dots, mn - 1\} \longrightarrow \{0, \dots, m - 1\} \times \{0, \dots, n - 1\}, \quad f(x) = (x \bmod m, x \bmod n).$$

Both sets have exactly mn elements, so f is a bijection if and only if it is injective. Suppose $f(x) = f(y)$ for $x, y \in \{0, \dots, mn - 1\}$. Then $x \equiv y \pmod{m}$ and $x \equiv y \pmod{n}$, so as in the uniqueness argument above, $mn \mid (x - y)$. But $|x - y| < mn$, so $x = y$. Hence f is injective, and since the domain and codomain are finite of equal size, f is a bijection.

In particular, every pair $(a \bmod m, b \bmod n)$ is hit by exactly one $x \in \{0, \dots, mn - 1\}$ — this x is simultaneously the existence and the uniqueness statement of the theorem. $\square$

In plain English: there are exactly mn numbers from 0 to $mn - 1$, and exactly mn possible pairs of remainders $(\bmod m, \bmod n)$. If two different numbers in that range could never collide (produce the same pair of remainders) — which is what coprimality guarantees — then, since the counts match exactly, every possible pair of remainders *must* be achieved by some number, and only one.

Example 3.3. Solve $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$. Here $m = 3, n = 5$. Bézout: $3(2) + 5(-1) = 1$, so $u = 2, v = -1$. Then $nv = 5(-1) = -5 \equiv 1 \pmod{3}, \equiv 0 \pmod{5}$, and $mu = 3(2) = 6 \equiv 0 \pmod{3}, \equiv 1 \pmod{5}$. So

$$x_0 = a \cdot nv + b \cdot mu = 2(-5) + 3(6) = -10 + 18 = 8.$$

Check: $8 = 2 \cdot 3 + 2$, remainder 2 $\pmod{3}$ ✓; $8 = 1 \cdot 5 + 3$, remainder 3 $\pmod{5}$ ✓. The full solution set is $x \equiv 8 \pmod{15}$.

4 The General Case: k Pairwise Coprime Moduli

Theorem 4.1 (CRT, general form). *Let $m_1, m_2, \dots, m_k$ be pairwise coprime positive integers (i.e. $\gcd(m_i, m_j) = 1$ for $i \neq j$), and let $M = m_1 m_2 \cdots m_k$. For any integers $a_1, \dots, a_k$, the system*

$$x \equiv a_1 \pmod{m_1}, \quad x \equiv a_2 \pmod{m_2}, \quad \dots, \quad x \equiv a_k \pmod{m_k}$$

has a solution, unique modulo M .

Proof. Induction on k . The base case $k = 1$ is trivial ($x \equiv a_1 \pmod{m_1}$ is already the full statement), and $k = 2$ is Theorem 3.1.

Inductive step. Suppose the theorem holds for $k - 1$ moduli. Given pairwise coprime $m_1, \dots, m_k$, first solve the subsystem in the first $k - 1$ congruences: by the inductive hypothesis, there is a unique y_0 modulo $M' := m_1 \cdots m_{k-1}$ satisfying

$$x \equiv a_i \pmod{m_i}, \quad i = 1, \dots, k - 1.$$

It remains to solve simultaneously

$$x \equiv y_0 \pmod{M'}, \quad x \equiv a_k \pmod{m_k}.$$

Key point: $\gcd(M', m_k) = 1$. Indeed, $M' = m_1 \cdots m_{k-1}$, and m_k is coprime to each m_i ($i < k$) by hypothesis; a number coprime to each of several integers is coprime to their product (no prime factor of m_k can divide M' , since such a prime would have to divide some m_i , contradicting $\gcd(m_i, m_k) = 1$). So $\gcd(M', m_k) = 1$, and Theorem 3.1 (two-modulus case) applies directly to this pair, giving existence and uniqueness of x modulo $M'm_k = M$.

This completes the induction. $\square$

In plain English: you don't need a fundamentally new idea for many moduli — you just fold them in one at a time. Solve the first two, combine them into a single congruence modulo their product, then combine that with the third modulus (which is still coprime to the product, since it was coprime to each factor separately), and so on. This is also exactly how you would implement CRT reconstruction as an algorithm.

Algorithm 4.2 (Iterative CRT reconstruction). Given pairwise coprime $m_1, \dots, m_k$ and targets $a_1, \dots, a_k$:

1. Initialize $x \leftarrow a_1, M \leftarrow m_1$.
2. For $i = 2, \dots, k$: find x' satisfying $x' \equiv x \pmod{M}$ and $x' \equiv a_i \pmod{m_i}$, using the two-modulus formula of Theorem 3.1 (Bézout construction) on the coprime pair (M, m_i) . Update $x \leftarrow x' \pmod{Mm_i}, M \leftarrow Mm_i$.
3. Return x (the unique solution modulo $M = m_1 \cdots m_k$).

Example 4.3 (Sunzi's Problem). Solve $x \equiv 2 \pmod{3}, x \equiv 3 \pmod{5}, x \equiv 2 \pmod{7}$.

Step 1. From Example 3.3, $x \equiv 2 \pmod{3}$ and $x \equiv 3 \pmod{5}$ combine to $x \equiv 8 \pmod{15}$.

Step 2. Combine $x \equiv 8 \pmod{15}$ with $x \equiv 2 \pmod{7}$. Bézout for $(15, 7)$: $15(1) + 7(-2) = 1$, so $u = 1, v = -2$ (here " m " = 15, " n " = 7). Then $nv = 7(-2) = -14 \equiv 1 \pmod{15}, \equiv 0 \pmod{7}$, and $mu = 15(1) = 15 \equiv 0 \pmod{15}, \equiv 1 \pmod{7}$. So

$$x_0 = 8 \cdot (-14) + 2 \cdot (15) = -112 + 30 = -82 \equiv -82 + 105 = 23 \pmod{105}.$$

Check: $23 = 7 \cdot 3 + 2 \pmod{3} \checkmark, 23 = 4 \cdot 5 + 3 \pmod{5} \checkmark, 23 = 3 \cdot 7 + 2 \pmod{7} \checkmark$.

So the smallest positive answer to Sunzi's ancient riddle is $x = 23$, with general solution $x \equiv 23 \pmod{105}$.

Example 4.4. Solve $x \equiv 1 \pmod{4}, x \equiv 2 \pmod{9}, x \equiv 3 \pmod{25}$ (moduli pairwise coprime, product = 900). Combine first two: need $x \equiv 1 \pmod{4}, x \equiv 2 \pmod{9}$. Try $x = 29$: $29 = 7 \cdot 4 + 1 \checkmark, 29 = 3 \cdot 9 + 2 \checkmark$. So $x \equiv 29 \pmod{36}$. Now combine with $x \equiv 3 \pmod{25}$: Bézout for $(36, 25)$: $36(-4) + 25(6) = -144 + 150 = 6 \dots$ adjust: try $36(4) + 25(-\dots)$. Use extended Euclid: $36 = 1 \cdot 25 + 11; 25 = 2 \cdot 11 + 3; 11 = 3 \cdot 3 + 2; 3 = 1 \cdot 2 + 1$. Back-substitute: $1 = 3 - 1 \cdot 2 = 3 - 1(11 - 3 \cdot 3) = 4 \cdot 3 - 11 = 4(25 - 2 \cdot 11) - 11 = 4 \cdot 25 - 9 \cdot 11 = 4 \cdot 25 - 9(36 - 25) = 13 \cdot 25 - 9 \cdot 36$. So $36(-9) + 25(13) = 1$, i.e. $u = -9, v = 13$ for $(m, n) = (36, 25)$. Then $nv = 25 \cdot 13 = 325 \equiv 1 \pmod{36}$ (since $325 = 9 \cdot 36 + 1$), $\equiv 0 \pmod{25}$; $mu = 36(-9) = -324 \equiv 0 \pmod{36}, \equiv 1 \pmod{25}$ (since $-324 = -13 \cdot 25 + 1$).

$$x_0 = 29(325) + 3(-324) = 9425 - 972 = 8453 \equiv 8453 - 9 \cdot 900 = 8453 - 8100 = 353 \pmod{900}.$$

Check: $353 = 88 \cdot 4 + 1 \checkmark; 353 = 39 \cdot 9 + 2 \checkmark; 353 = 14 \cdot 25 + 3 \checkmark$. So $x \equiv 353 \pmod{900}$.

5 The Algebraic Formulation: A Ring Isomorphism

The deepest and most reusable version of CRT is not a statement about solving congruences, but a statement about the *structure* of the ring $\mathbb{Z}/M\mathbb{Z}$ itself.

Theorem 5.1 (CRT as ring isomorphism). *Let $m_1, \dots, m_k$ be pairwise coprime, $M = m_1 \cdots m_k$. The map*

$$\Psi : \mathbb{Z}/M\mathbb{Z} \longrightarrow \mathbb{Z}/m_1\mathbb{Z} \times \mathbb{Z}/m_2\mathbb{Z} \times \cdots \times \mathbb{Z}/m_k\mathbb{Z}, \quad \Psi(x \bmod M) = (x \bmod m_1, \dots, x \bmod m_k)$$

is a ring isomorphism: it is a bijection that respects both addition and multiplication,

$$\Psi(x + y) = \Psi(x) + \Psi(y), \quad \Psi(xy) = \Psi(x)\Psi(y),$$

where addition and multiplication on the right side are done componentwise.

Proof. Well-defined: if $x \equiv x' \pmod{M}$ then $x \equiv x' \pmod{m_i}$ for each i (since $m_i \mid M$), so Ψ does not depend on the choice of representative x .

Bijjective: this is exactly Theorem 4.1 — every tuple $(a_1, \dots, a_k)$ arises from a unique $x \bmod M$, so Ψ is a bijection.

Respects addition: $(x + y) \bmod m_i = ((x \bmod m_i) + (y \bmod m_i)) \bmod m_i$ for each i — this is just the elementary fact that reduction mod m_i is compatible with addition. Hence $\Psi(x + y) = \Psi(x) + \Psi(y)$ componentwise.

Respects multiplication: identically, $(xy) \bmod m_i = ((x \bmod m_i)(y \bmod m_i)) \bmod m_i$ for each i , so $\Psi(xy) = \Psi(x)\Psi(y)$.

A bijection compatible with both ring operations is by definition a ring isomorphism. $\square$

In plain English: arithmetic mod M is *secretly the same thing* as doing arithmetic mod m_1 , mod m_2 , $\dots$, mod m_k all simultaneously and independently, then reassembling the answer. You can add, subtract, or multiply numbers coordinate-by-coordinate in the separate small moduli, and it always agrees with doing the operation directly in the big modulus M . This is not just a coincidence about congruences; it is a full structural equivalence of algebraic systems (rings).

Corollary 5.2 (Units transfer). *Under Ψ , an element $x \bmod M$ is a unit (invertible under multiplication) in $\mathbb{Z}/M\mathbb{Z}$ if and only if each component $x \bmod m_i$ is a unit in $\mathbb{Z}/m_i\mathbb{Z}$. Consequently,*

$$(\mathbb{Z}/M\mathbb{Z})^\times \cong (\mathbb{Z}/m_1\mathbb{Z})^\times \times \cdots \times (\mathbb{Z}/m_k\mathbb{Z})^\times$$

as groups (in fact, as multiplicative monoids restricted to units), and taking cardinalities recovers

$$\varphi(M) = \varphi(m_1)\varphi(m_2)\cdots\varphi(m_k),$$

i.e. the multiplicativity of Euler's totient function is a direct corollary of the CRT ring isomorphism.

Proof. A ring isomorphism sends units to units and non-units to non-units (an inverse in one ring corresponds, under the isomorphism, to an inverse in the other — apply Ψ to the equation $xy = 1$). Units in a product ring $R_1 \times \cdots \times R_k$ are exactly tuples that are units in *every* coordinate (invert coordinatewise). Restricting the bijection Ψ to unit groups gives the stated group isomorphism, and cardinalities multiply because the cardinality of a Cartesian product is the product of cardinalities. $\square$

This is precisely the argument used (in more computational language) in the earlier lecture on Euler's totient function — CRT is the machine underneath multiplicativity of φ .

6 CRT with Non-Coprime Moduli

What if the moduli are *not* pairwise coprime? The system may or may not have a solution, and when it does, the modulus of the combined solution shrinks from the product to the least common multiple.

Theorem 6.1 (Generalized CRT). *Let m, n be positive integers (not necessarily coprime) and $a, b \in \mathbb{Z}$. The system*

$$x \equiv a \pmod{m}, \quad x \equiv b \pmod{n}$$

has a solution if and only if

$$a \equiv b \pmod{\gcd(m, n)}.$$

When a solution exists, it is unique modulo $\text{lcm}(m, n)$.

Proof. Let $g = \gcd(m, n)$, $M = \text{lcm}(m, n)$.

Necessity. If x solves both congruences, then $x = a + sm = b + tn$ for integers s, t , so $a - b = tn - sm$. Since $g \mid m$ and $g \mid n$, the right side is divisible by g , so $g \mid (a - b)$, i.e. $a \equiv b \pmod{g}$.

Sufficiency and construction. Suppose $a \equiv b \pmod{g}$, i.e. $g \mid (b - a)$. By Bézout, there exist u, v with $mu + nv = g$. Then

$$b - a = \left(\frac{b - a}{g}\right) g = \left(\frac{b - a}{g}\right) (mu + nv).$$

Set $x_0 = a + mu \cdot \frac{b - a}{g}$. Then $x_0 \equiv a \pmod{m}$ trivially (we added a multiple of m). Also

$$x_0 = a + \frac{(b - a)}{g}(g - nv) = a + (b - a) - \frac{(b - a)}{g}nv = b - \frac{(b - a)}{g}nv \equiv b \pmod{n},$$

since we subtracted a multiple of n . So x_0 solves the system.

Uniqueness modulo $M = \text{lcm}(m, n)$. If x_0, x_1 both solve the system then $m \mid (x_0 - x_1)$ and $n \mid (x_0 - x_1)$, so $\text{lcm}(m, n) \mid (x_0 - x_1)$ (the lcm is, by definition, the smallest — and in fact the unique minimal generator of the set of — common multiples; any common multiple of m, n is a multiple of $\text{lcm}(m, n)$, a standard fact from the prime-factorization description of lcm and gcd). Hence $x_0 \equiv x_1 \pmod{M}$. $\square$

In plain English: if m, n share a common factor g , then knowing $x \pmod{m}$ already pins down $x \pmod{g}$, and knowing $x \pmod{n}$ also pins down $x \pmod{g}$ — so for the system to be consistent at all, these two implied values mod g must agree. If they do agree, there's no further obstruction, and the combined information determines x up to the least common multiple (the smallest modulus consistent with both individual congruences) rather than the full product mn (which would double-count the shared factor g).

Example 6.2. Solve $x \equiv 4 \pmod{6}$, $x \equiv 1 \pmod{9}$. Here $g = \gcd(6, 9) = 3$. Check consistency: $4 \pmod{3} = 1$, $1 \pmod{3} = 1$ — they agree, so a solution exists. $M = \text{lcm}(6, 9) = 18$. Try $x = 10$: $10 = 1 \cdot 6 + 4\checkmark$, $10 = 1 \cdot 9 + 1\checkmark$. So $x \equiv 10 \pmod{18}$.

By contrast, $x \equiv 2 \pmod{4}$, $x \equiv 1 \pmod{6}$ has *no* solution: $g = \gcd(4, 6) = 2$, and $2 \pmod{2} = 0$ while $1 \pmod{2} = 1$ — inconsistent.

7 The Polynomial Chinese Remainder Theorem

CRT is not special to $\mathbb{Z}$; it holds in any **principal ideal domain**, and the polynomial ring $F[x]$ over a field F is a particularly important example, because it directly explains **Lagrange interpolation**.

Theorem 7.1 (CRT for polynomials). *Let F be a field and $p_1(x), \dots, p_k(x) \in F[x]$ be pairwise coprime polynomials (no common roots/factors), with $P(x) = p_1(x) \cdots p_k(x)$. For any polynomials $a_1(x), \dots, a_k(x)$, there is a polynomial $g(x)$, unique modulo $P(x)$, satisfying*

$$g(x) \equiv a_i(x) \pmod{p_i(x)}, \quad i = 1, \dots, k.$$

Proof sketch. Identical in structure to Theorem 4.1: because $F[x]$ has a Euclidean algorithm (polynomial long division), Bézout’s identity holds for coprime polynomials, $p_i(x)u(x) + p_j(x)v(x) = 1$, and the same construction and induction go through verbatim, replacing integers with polynomials throughout. $\square$

Corollary 7.2 (Lagrange Interpolation as a special case). *Take $p_i(x) = x - c_i$ for k distinct points $c_1, \dots, c_k \in F$ (these are pairwise coprime linear polynomials since they have distinct roots), and let $a_i(x) = y_i$ (constants) be target values. The unique polynomial $g(x)$ of degree $< k$ with $g(c_i) = y_i$ for all i (interpolation through the k points (c_i, y_i)) is precisely the CRT reconstruction, and it recovers the classical Lagrange interpolation formula*

$$g(x) = \sum_{i=1}^k y_i \prod_{j \neq i} \frac{x - c_j}{c_i - c_j}.$$

In plain English: “knowing the remainder of $g(x)$ modulo $(x - c_i)$ ” is exactly the same as “knowing the value $g(c_i)$ ” (by the polynomial remainder theorem). So reconstructing a polynomial from its values at several points (interpolation) and reconstructing an integer from its remainders modulo several numbers (CRT) are, structurally, *the exact same theorem*, just applied in two different rings ($F[x]$ versus $\mathbb{Z}$). This is a beautiful example of how abstract algebra unifies superficially unrelated classical results.

8 Applications

8.1 Multiplicativity of Euler’s totient function

As shown in Corollary 5.2, $\varphi(mn) = \varphi(m)\varphi(n)$ for coprime m, n is an immediate consequence of the CRT ring isomorphism restricted to unit groups. This is the same fact used (with an equivalent but more computational proof) in the companion lecture on the Euler totient function.

8.2 Fast RSA decryption: Garner’s algorithm / CRT-RSA

In RSA with modulus $n = pq$, a private-key holder who knows p, q separately can decrypt *roughly four times faster* using CRT:

- Compute $m_p = c^{d \bmod (p-1)} \bmod p$ and $m_q = c^{d \bmod (q-1)} \bmod q$ — two *much smaller* modular exponentiations (roughly half the bit-length each, and exponentiation cost grows worse than linearly in bit-length).
- Recombine m_p, m_q into $m \bmod n$ using the CRT formula of Theorem 3.1 (a fixed procedure known as **Garner’s algorithm**, which precomputes the Bézout-type coefficients once per key).

This CRT-based speedup is used in essentially every real-world RSA implementation.

8.3 Secret sharing (Asmuth–Bloom scheme)

CRT-based secret sharing splits a secret S into k shares such that any t of them can reconstruct S , but fewer cannot. Roughly: choose pairwise coprime moduli $m_1 < m_2 < \dots < m_k$ such that the product of any t of them exceeds S , but the product of any $t - 1$ of them does not. The secret is encoded as $S \bmod (m_1 \cdots m_t)$ -type residues, distributed as shares $S \bmod m_i$; any t shares let you reconstruct $S \pmod{\text{the product of those } t \text{ moduli}}$, which is large enough to pin S down exactly) via CRT, while $t - 1$ shares leave genuine ambiguity.

8.4 Computer arithmetic: Residue Number Systems (RNS)

Large-integer arithmetic (as used in cryptography and signal processing) can represent a big number x as a tuple of remainders $(x \bmod m_1, \dots, x \bmod m_k)$ for pairwise coprime word-sized m_i . Addition, subtraction, and multiplication of the big number can then be done *independently and in parallel* in each small modulus (Theorem 5.1), which is far faster on hardware than doing the arithmetic directly on the large number, at the cost of needing CRT reconstruction (or avoiding it entirely, if only sums/products of residues are needed later) when a normal decimal/binary representation is required.

8.5 Fast algorithms: multiplying big numbers, FFT-based convolution

Some fast multiplication algorithms (and computations of convolutions, relevant to signal processing and polynomial multiplication) use CRT — either in $\mathbb{Z}$ over several prime moduli (to avoid overflow, then reconstructing at the end) or via the polynomial CRT of Theorem 7.1 (splitting a polynomial multiplication problem into evaluation at several points, multiplying pointwise, then interpolating back — this is precisely the philosophy behind the Fast Fourier Transform’s evaluate–multiply–interpolate strategy, where the “points” are roots of unity).

8.6 Error-correcting codes

Certain codes (e.g., Redundant Residue Number System codes) encode data redundantly using more moduli than strictly necessary for reconstruction; if a small number of residues are corrupted, CRT-like reconstruction from the remaining consistent residues can detect and correct the error, analogous to redundancy-based error correction in other coding schemes.

9 Summary of Key Formulas

Statement	Formula / Content
Two-modulus CRT	$x \equiv a \pmod{m}, x \equiv b \pmod{n}, \gcd(m, n) = 1 \Rightarrow \text{unique } x \pmod{mn}$
Bézout construction	$mu + nv = 1 \Rightarrow x_0 = anv + bmu$
Direct formula (Method B)	$x \equiv \sum_i a_i M_i y_i \pmod{M}, M_i = M/m_i, M_i y_i \equiv 1 \pmod{m_i}$
Two-at-a-time (Method C)	$x \equiv a + m[(b - a)m^{-1} \pmod{n}] \pmod{mn}$
General CRT	pairwise coprime $m_1, \dots, m_k \Rightarrow \text{unique } x \pmod{M = \prod m_i}$
Ring isomorphism	$\mathbb{Z}/M\mathbb{Z} \cong \mathbb{Z}/m_1\mathbb{Z} \times \dots \times \mathbb{Z}/m_k\mathbb{Z}$
Units transfer	$(\mathbb{Z}/M\mathbb{Z})^\times \cong \prod_i (\mathbb{Z}/m_i\mathbb{Z})^\times$
Totient corollary	$\varphi(M) = \prod_i \varphi(m_i)$
Non-coprime case	solvable $\iff a \equiv b \pmod{\gcd(m, n)}$; unique $\pmod{\text{lcm}(m, n)}$
Polynomial CRT	pairwise coprime $p_i(x) \Rightarrow \text{unique } g(x) \pmod{\prod p_i(x)}$
Lagrange interpolation	special case with $p_i(x) = x - c_i$

10 Exercises

- (1) Solve $x \equiv 3 \pmod{7}, x \equiv 5 \pmod{11}, x \equiv 7 \pmod{13}$ using the iterative algorithm (Algorithm 4.2).
- (2) Determine whether $x \equiv 5 \pmod{12}, x \equiv 1 \pmod{18}$ has a solution; if so, find all solutions.
- (3) Prove that if $m_1, \dots, m_k$ are pairwise coprime and each $m_i \geq 2$, then the number of solutions to $x^2 \equiv 1 \pmod{M}$ (with $M = \prod m_i$) equals 2^k when every m_i is odd (relate this to the unit-group isomorphism of Corollary 5.2 and the number of square roots of 1 in each cyclic-or-not factor).
- (4) Using the polynomial CRT, reconstruct the unique polynomial of degree < 3 with $g(0) = 1, g(1) = 2, g(2) = 5$, and verify it matches the Lagrange interpolation formula.
- (5) Explain, in your own words, why CRT-RSA decryption is faster, using the fact that modular exponentiation cost grows more than linearly in the bit-length of the modulus.
- (6) Show that $\mathbb{Z}/6\mathbb{Z} \cong \mathbb{Z}/2\mathbb{Z} \times \mathbb{Z}/3\mathbb{Z}$ explicitly, by writing out the full addition and multiplication tables on both sides and matching them via Ψ .